PROJECT REPORT

Lecture Notes

Real-time AI transcription, speaker diarization & insights — deployed on AWS

Author: Dev Agarwal

Programme: MCA, Nirma University

Date: October 2026

Stack: Next.js · Mistral Voxtral · PostgreSQL · AWS

Next.js 16 | React 19 | TypeScript | Prisma | Mistral Voxtral | Amazon RDS | Amazon S3 | AWS Lambda | Amazon SQS

Contents

1. Introduction and Objectives
2. System Overview and Architecture
3. Technology Stack
4. Data Model
5. The AI Pipeline (Mistral Voxtral)
6. End-Word Line Control
7. Cloud Architecture on AWS
8. Security and Configuration
9. Testing and Quality Assurance
10. Results and Measurements
11. Limitations and Future Work
12. Conclusion

1. Introduction and Objectives

Lecture Notes is a web application that lets a professor record a lecture and have it turned into an organised, searchable set of notes. The application captures audio from the browser microphone, transcribes it live on screen, and — once the recording stops — produces a speaker-labelled transcript together with AI-generated insights (a short summary, key points, and topic tags). Recordings are organised on a calendar and can be searched by title.

The project was built to explore a complete, production-shaped pipeline: a modern full-stack web application, a hosted speech-to-text and summarisation layer, and a cloud deployment on AWS with object storage, a managed database, secret management, and an asynchronous worker.

1.1 Objectives

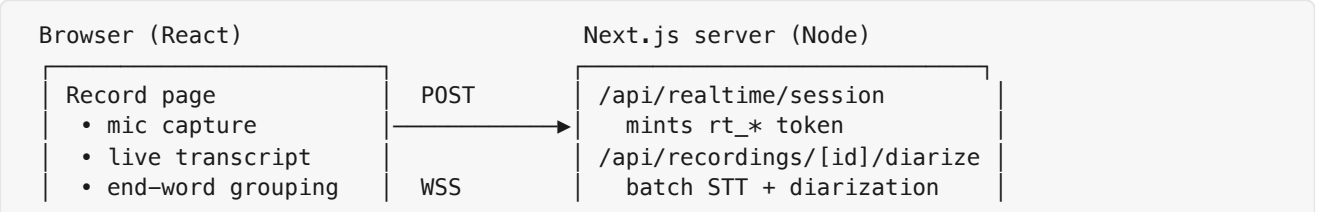
- Capture lecture audio in the browser and transcribe it in real time.
- Produce a final, high-quality transcript with speaker diarization.
- Group transcript lines under the speaker's control using configurable *end words* rather than fixed time intervals.
- Generate a summary, key points, and topics from the transcript.
- Persist recordings, transcripts, speakers, and insights durably.
- Deploy on AWS with S3, RDS, Secrets Manager, SQS and Lambda — and keep the provider API key strictly server-side.
- Back the system with automated unit, API, and browser end-to-end tests.

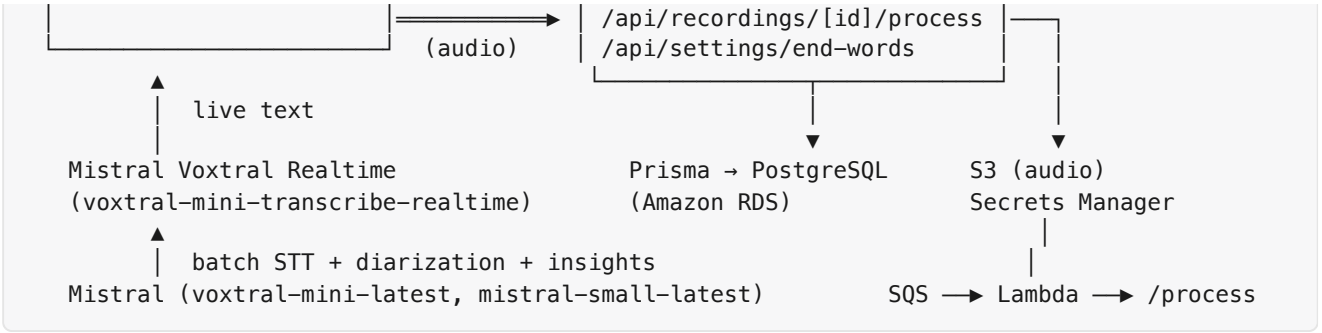
1.2 Scope

The deliverable is a single Next.js application containing both the user interface and the server (API routes), backed by PostgreSQL, with recorded audio stored in Amazon S3 and processed through a Mistral Voxtral model family for speech-to-text and Mistral Small for summarisation. The application is containerised and runs on AWS EC2 (with an SQS-triggered Lambda worker), and the infrastructure is described as code.

2. System Overview and Architecture

The application follows a conventional three-tier layout — a React client, a Next.js server, and a PostgreSQL database — augmented by two external AI services and a set of AWS managed services. The browser talks directly to Mistral's realtime endpoint for live transcription (using a short-lived token), while all privileged operations happen on the server.





2.1 Primary User Flows

1. **Record.** The user opens `/dashboard/record`, optionally names the lecture, and starts recording. The browser requests microphone access, opens a WebSocket to Mistral, and streams 16-bit PCM.
2. **Live transcription.** Word-level text deltas arrive over the socket and are accumulated into the current line. When an end word is heard (or a periodic fallback fires), the line is committed.
3. **Finalisation.** On stop, the full audio is sent to the server, which re-transcribes it with diarization, re-groups it into end-word lines, assigns speakers, stores the audio in S3, and returns the finished recording.
4. **Insights.** The transcript is summarised by an LLM and the resulting summary, key points and topics are stored.
5. **Review.** The transcript and insights pages display the recording; audio can be played back through a presigned S3 URL.

3. Technology Stack

Layer	Technology	Notes
Framework	Next.js 16 (App Router, Turbopack)	Server + client in one codebase
UI	React 19, TypeScript	Typed components and hooks
Styling	Tailwind CSS v4, shadcn/ui, lucide-react, framer-motion	Minimal, monochrome design
Data access	Prisma ORM + `@prisma/adapter-pg`	Type-safe queries
Database	PostgreSQL (Amazon RDS)	Driver-adapter based client
Speech-to-text	Mistral Voxtral	Realtime + batch with diarization
Summarisation	Mistral Small	Streamed chat completion

Layer	Technology	Notes
Object storage	Amazon S3	Recorded audio, presigned reads
Secrets	AWS Secrets Manager	Mistral key, database URL
Async worker	Amazon SQS + AWS Lambda	Decoupled processing
Compute	AWS EC2 (container via ECR)	Next.js standalone image
Runtime / tooling	Bun, Docker, Playwright	Install, image, E2E tests

4. Data Model

Persistence uses Prisma with five models. A recording owns its transcript segments, its speakers, and a single insights row; deleting a recording cascades to all of them. Configured end words are stored in a small global table.

Model	Key fields	Purpose
Recording	id, title, date, duration, status, audioPath	A lecture session
TranscriptSegment	text, timestamp, startTime, endTime, languageCode, speakerId	One transcript line
Speaker	label, name, color	Diarized speaker
Insights	summary, keyPoints, topics, wordCount	AI-generated notes
EndWord	word (unique)	Line-end trigger words

```
Recording 1──* TranscriptSegment *──1 Speaker        (speaker optional)
Recording 1──1 Insights
EndWord   (global list, no relation)
```

Relations use `onDelete: Cascade` from Recording, so removing a lecture cleans up its segments and speakers automatically. Indexes on `recordingId`, `speakerId`, and `date` keep the list and detail queries fast.

5. The AI Pipeline (Mistral Voxtral)

Three models are used, each chosen for a specific role.

Role	Model	Endpoint
Live transcription	`voxtral-mini-transcribe-realtime-2602`	WebSocket
Batch transcription + diarization	`voxtral-mini-latest`	`POST /v1/audio/transcriptions`
Summarisation / insights	`mistral-small-latest`	`POST /v1/chat/completions` (streamed)

5.1 Live Transcription

Because browsers cannot store a long-lived API key or set authentication headers on a WebSocket, the server mints a short-lived `rt_*` token scoped to the realtime model. The browser opens the socket using the token as a sub-protocol, then streams base64-encoded 16-bit PCM at 16 kHz. The server never exposes the raw key to the client.

5.2 Batch Diarization

When recording stops, the accumulated screenshot-quality audio is sent to the server and re-transcribed with `diarize: true` and segment-level timestamps. This yields segments that carry a speaker id and time range, which are mapped to stored speakers. The result replaces the live (lower-quality) segments.

5.3 Insights

The final transcript is streamed to Mistral Small with a strict instruction to return JSON containing a summary, key points, and topics. The server streams the model output back to the browser, so the summary appears progressively, and the parsed result is persisted.

6. End-Word Line Control

By default, the live transcript commits a line every few seconds, producing a new line for almost every sentence. The application instead lets the professor control line breaks with **end words**: a line is finalised only when one of the configured words or phrases is spoken, and everything said in between accumulates into the current line.

Behaviour

- End words are managed on a dedicated **Settings** page (add / remove, stored in the database).
- Matching is case-insensitive and whole-word (so `done` does not match inside `abandoned`); multi-word phrases are supported.
- The trigger word is treated as a command and removed from the transcript; text after it begins the next line.

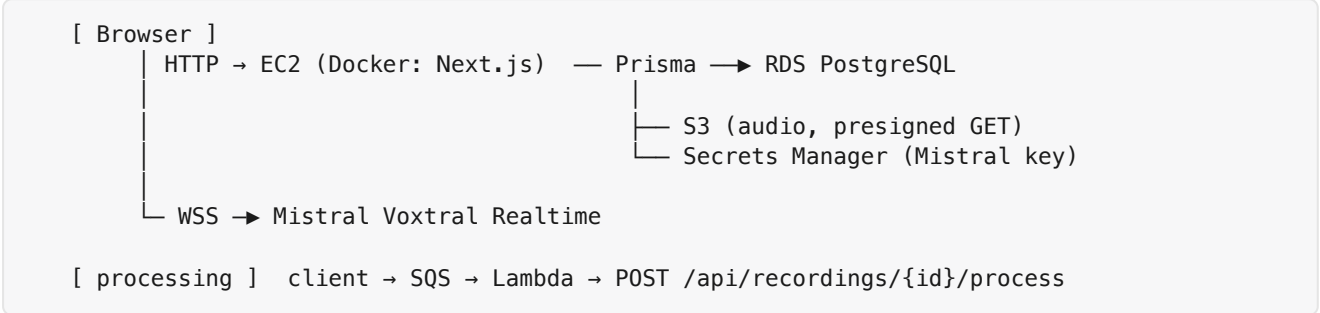
- The same rule is applied to the final diarized transcript, so stored line breaks and speaker labels share one source of truth.
- With no end words configured, the application falls back to the previous periodic behaviour, so nothing regresses.

```
Spoken:   "...we finished the topic done let us move on..."
Line 1:   "...we finished the topic"
Line 2:   "let us move on..."   (continues until the next end word)
```

7. Cloud Architecture on AWS

The application is deployed on AWS. The table lists the services and their role; the diagram shows how they connect.

AWS service	Role in this project
Amazon EC2 (t4g.small)	Runs the containerised Next.js app behind port 80
Amazon ECR	Stores the Docker image of the app
Amazon RDS for PostgreSQL	Managed database for Prisma
Amazon S3	Stores recorded audio; playback via presigned URLs
AWS Secrets Manager	Holds the Mistral API key and the database URL
Amazon SQS	Queue of recordings awaiting processing
AWS Lambda	Worker that consumes the queue and triggers processing
Amazon CloudWatch	Logs for the Lambda worker
IAM (LabRole / LabInstanceProfile)	Instance and Lambda permissions



7.1 Deployment

The app is built into a self-contained Next.js “standalone” server and shipped as a Docker image to ECR. An EC2 instance pulls the image and runs it; the instance profile supplies temporary credentials so the app can read S3 and Secrets Manager without any static keys. Because the lab account denies `iam:CreateRole` (which App Runner and Fargate require), EC2 with the pre-existing

instance profile is used as the compute tier. A reference SST configuration and a reproducible provisioning script accompany the code.

8. Security and Configuration

- **No API key in the browser.** The Mistral key is read only on the server, from Secrets Manager in the cloud and from the environment locally. The client receives short-lived realtime tokens instead.
- **Least privilege:** the instance role is scoped to the two secrets, the audio bucket, and the queue.
- **Audio access:** S3 objects are private; playback uses time-limited presigned URLs.
- **Worker authentication:** the async processing endpoint requires a shared secret header, so only the Lambda can trigger it.
- **Transport:** database connections use TLS; credentials and secrets are never committed to the repository.

Environment variables: `DATABASE_URL`, `AUDIO_BUCKET`, `AWS_REGION`, `MISTRAL_SECRET_ARN`, optional `SQS_QUEUE_URL` and `PROCESS_SECRET`.

9. Testing and Quality Assurance

The project is backed by an automated test suite of **38 tests** across three layers, run with Bun's test runner. The API and end-to-end suites run against the real application, database, object store, and AI services.

Suite	What it covers	Tests
Unit	Audio/WAV/base64 helpers, language normalisation, Mistral client, end-word split/group	13
API	Recordings CRUD, segments/speakers, end-word config, realtime token mint, diarization, insights, realtime WebSocket protocol	24
E2E (browser)	Headless Chromium with a fake microphone: record → live transcript → stop → diarization + insights → transcript page	1

Static analysis and builds are also part of the pipeline: ESLint runs with the Next.js ruleset (no errors or warnings), and the production build (`next build`) type-checks the entire project, including the tests. The browser suite uses a generated speech fixture played as the microphone input, so it exercises the real recording path deterministically.

10. Results and Measurements

The application runs end to end on AWS and was verified against the live deployed instance: realtime token minting via Secrets Manager, transcription with diarization, audio persistence to S3

with presigned playback, and the SQS → Lambda pipeline (CloudWatch recorded a successful processing run). All 38 tests pass against the cloud database and object store.

10.1 Transcription Length Probe

A measurement script pushed progressively longer audio through the batch endpoint to observe behaviour and limits. Results (single speaker, no rate limit encountered):

Audio length	Status	Latency	Tokens
1 minute	200 OK	~63 s	15,634
5 minutes	200 OK	~67 s	30,937
15 minutes	200 OK	~83 s	137,646
30 minutes	in progress (upload-bound)	—	—

The provider documents a per-request limit of up to three hours for batch transcription; the probe confirmed multi-minute requests succeed with sub-linear latency growth.

11. Limitations and Future Work

- **Compute tier.** App Runner / ECS Fargate were blocked by the lab's IAM policy; a full account would host the container behind a load balancer with TLS and a custom domain.
- **Async wiring.** The SQS + Lambda pipeline and the `/process` endpoint exist and are verified; the recorder still commits the final transcript synchronously. Switching the client to enqueue work is a small, isolated change.
- **Authentication.** The app currently has no user accounts; Cognito would add per-user recordings and sharing.
- **Search.** Only title search exists; embeddings and a vector store would enable semantic search across lectures.
- **Citation of the provider.** Transcription is billed per minute and summarisation per token; a cost dashboard would help tune the length of batch jobs.

12. Conclusion

Lecture Notes demonstrates a complete, cloud-native voice-notes application. It captures lecture audio in the browser, transcribes it live and again with speaker diarization, lets the speaker control transcript line breaks with configurable end words, and generates a summary with key points and topics. The system is deployed on AWS using S3, RDS, Secrets Manager, SQS and Lambda, with the AI provider key kept strictly server-side and the audio secured behind presigned URLs. A 38-test suite — unit, API, and real browser end-to-end — keeps the pipeline honest. The result is a practical, well-tested foundation that extends naturally towards authentication, richer search, and a fully load-balanced production deployment.

Appendix — API Routes

Method & path	Purpose
`POST /api/realtime/session`	Mint a short-lived Voxtral realtime token
`GET/POST /api/recordings`	List / create recordings
`GET/PATCH/DELETE /api/recordings/[id]`	Read / update / delete a recording
`POST /api/recordings/[id]/diarize`	Transcribe uploaded audio with diarization
`POST /api/recordings/[id]/process`	Async processing from stored audio (Lambda)
`GET /api/recordings/[id]/audio`	Presigned S3 redirect for playback
`POST/GET /api/recordings/[id]/insights`	Generate / read insights
`POST /api/recordings/[id]/speakers`	Manage speakers
`POST /api/recordings/[id]/segments`	Persist transcript segments
`GET /api/recordings/by-date`	Group recordings for the calendar
`GET/POST/DELETE /api/settings/end-words`	Manage end words